Chat com ordenação

Documento de protocolo – v1

Levi Henri André Gabriel

5 de novembro de 2026

Estado. A v1 fecha o contrato da operação normal, do heartbeat, do Bully e do bootstrap (seções 2 e 4), implementados em `app/` e testados localmente (seção 5). A reorganização e a reentrada (seção 3) continuam como proposta, a implementar em T05 e T06. O texto ainda depende da revisão do grupo. Arquitetura e justificativas ficam em `docs/relatorio.tex`.

1 Escopo e transporte

Três nós equivalentes, identificados por 1, 2 e 3, trocam JSON por HTTP. Clientes usam WebSocket. Nomes dos serviços vêm da configuração e são resolvidos pelo DNS do Compose; não há IPs fixos no código. O cenário inicial é uma sala, com estado em memória.

A recuperação proposta pressupõe uma queda por vez, rede funcionando entre os sobreviventes e nenhuma nova falha durante reorganização ou sincronização. São necessários dois dos três nós para operar. O relatório detalha os limites.

2 Operação normal

Identidade e visão. Cada nó tem ID 1, 2 ou 3. Uma visão contém época, líder e membros ativos. A época é proposta como o par (contador, ID do coordenador), que distingue propostas sem substituir acordo entre participantes. Estados locais: sincronizando, ativo e reorganizando.

Cada nó mantém relógio L, fila de pendências, ACKs recebidos, histórico, UUIDs entregues e identificação dos pacotes já processados. O histórico preserva o conteúdo, a época e o carimbo original de cada evento, necessários para a reconciliação; guardar apenas UUIDs não basta.

Cliente. Envia `id`, `sala`, `autor` e `texto`; `id` é um UUID preservado nos reenvios. Reutilizar um UUID com conteúdo diferente é erro de protocolo.

Envelope entre nós. Campos comuns: `tipo`, `pacote_id`, `epoca` e `origem`. Um retry preserva o mesmo pacote. `MSG` acrescenta relógio e conteúdo da mensagem; `ACK` acrescenta seu relógio e a referência ao evento reconhecido. O evento é identificado pela época, origem e timestamp original.

1. Ao aceitar uma mensagem do cliente, o nó incrementa L, registra o evento e enfileira seu multicast para todos os membros, incluindo recepção local.
2. Ao receber `MSG`, atualiza $L \leftarrow \max(L, t) + 1$ e guarda a mensagem na fila, ordenada pelo **carimbo original** (t, origem). O relógio atualizado do receptor não altera essa chave.
3. O receptor incrementa seu relógio para emitir `ACK` a todos os membros. Receber um ACK também atualiza L pela regra de recepção. ACKs que chegam antes do MSG correspondente são guardados.
4. A primeira mensagem da fila só é entregue após ACK de todos os membros daquela visão. O UUID é registrado no histórico e os clientes locais são avisados.

Confirmação. O nó do cliente confirma a mensagem quando a entrega. Receber pelo WebSocket ou responder a um POST não confirma entrega. Na reconexão, o cliente reenvia usando o mesmo UUID; o reenvio também serve de consulta. O contrato com o navegador está na seção 4.

Relógio. Modela os eventos da camada `MSG/ACK`. Heartbeats ficam fora dessa ordenação. Dentro de uma visão, a chave é (t, origem); entre visões, o histórico anterior deve permanecer como prefixo imutável. Lamport ordena mensagens; o líder coordena a visão.

FIFO, retries e deduplicação

Uma fila de saída por destino envia um POST por vez e só avança após a resposta. Falhas de transporte repetem o mesmo pacote. Uma resposta pode se perder depois do processamento: o receptor deve responder ao retry sem repetir seus efeitos.

Atualização de estado e enfileiramento de cada multicast são serializados. O multicast inteiro é enfileirado antes de processar outro evento. O handler não espera respostas de outros nós para concluir o processamento local.

A deduplicação ocorre no transporte e na entrega. Pacotes repetidos não geram novo evento nem nova emissão de ACK; um UUID entregue não reaparece no histórico.

Identidade após reinício. O `pacote_id` tem a forma `<origem>-<encarnação>-<sequência>`. A encarnação é sorteada a cada início do processo; um nó reiniciado nunca repete um identificador antigo.

Mesmo UUID em nós distintos. Se o cliente reenviar a mesma tentativa a outro nó antes da entrega, podem existir dois eventos com o mesmo UUID. Ambos seguem a ordem total, e todos os nós entregam somente a primeira ocorrência, descartando as seguintes na entrega. Como todos veem a mesma sequência, todos descartam a mesma ocorrência. Dentro de um nó, UUID repetido com conteúdo igual não cria novo evento; com conteúdo diferente é recusado.

Heartbeat e eleição

Parâmetros padrão: heartbeat a cada 1 segundo e suspeita após 3 segundos sem heartbeat; espera de `OK` de 1 segundo e de `COORDINATOR` de 3 segundos. São configuráveis e não representam prazo garantido de recuperação. Heartbeats vão a todos os nós configurados, inclusive fora da visão.

A suspeita de qualquer membro passa o nó a *reorganizando*: novos envios de clientes são recusados, enquanto MSG e ACK da visão atual continuam sendo processados até a pausa da transição. Com líder disponível, ele coordena a reorganização. Se o coordenador for suspeito, os membros com visão executam Bully com `ELECTION`, `OK` e `COORDINATOR`: `ELECTION` vai aos membros de ID maior não suspeitos; sem `OK`, o nó se anuncia; com `OK` e sem anúncio dentro do prazo, repete a eleição. Quem recebe `ELECTION` de ID menor responde `OK` e inicia a própria eleição. Um anúncio de ID menor recebido por um nó vivo de ID maior provoca nova eleição.

Mensagens de eleição de outra época são ignoradas. Nós sincronizando não respondem nem votam. Vencer só define o coordenador da próxima transição: a visão e a época permanecem, e os envios continuam pausados até a retomada. Suspeitas são permanentes até a próxima instalação de visão.

3 Reorganização e reentrada: proposta a revisar

A transição fecha a visão antiga antes de retomar a aplicação. A sequência abaixo é um esboço de recuperação sob as hipóteses da seção 1; sua segurança ainda deve ser examinada em exemplos e testes.

O que significa o corte. Visão é a configuração compartilhada de membros, líder e época. O corte é a pausa local que separa o processamento normal da visão antiga e a coleta do estado para reorganização. Não é um horário global nem um limite no relógio de Lamport: cada nó pausa em um momento diferente. A pausa e a captura de estado são serializadas com o processamento local. Uma

mensagem aceita antes da pausa aparece no histórico ou nas pendências informadas; depois da pausa, MSGs e ACKs antigos deixam de alterar esse estado.

1. `VIEW_PREPARE`: o coordenador informa a transição e os membros propostos. O próprio coordenador também executa a pausa e participa da coleta.
2. Cada participante pausa novos envios e entregas, encerra retries de `MSG`/`ACK` da visão antiga e captura seu estado. Cópias antigas desses dois tipos recebidas depois da pausa são descartadas com resposta explícita, encerrando o retry. Os controles válidos da transição continuam sendo processados; a regra de descarte não pode bloquear a reorganização.
3. `VIEW_STATE`: todos os membros propostos informam histórico, pendências e relógio. O coordenador aguarda todos.
4. O coordenador reúne os eventos conhecidos, ordenando os da visão encerrada pelos carimbos originais. O resultado deve preservar, como prefixo, cada histórico já entregue. Incompatibilidade mantém a pausa e sinaliza erro.
5. `VIEW_INSTALL`: distribui nova época, membros, histórico reconciliado e relógio de referência, ao menos o maior relógio informado na coleta. Os nós não reduzem seu relógio local. Instalam o mesmo estado e permanecem pausados, preservando o sufixo ainda não notificado aos seus clientes.
6. `VIEW_READY`: cada membro confirma a instalação. Só após todos confirmarem, o coordenador envia `VIEW_RESUME`. Na retomada, cada nó notifica seus clientes somente das mensagens que faltavam em seu histórico anterior, preservando a ordem e os UUIDs já entregues.

Mensagens da nova visão recebidas antes da retomada aguardam processamento. Pacotes antigos não retornam à operação normal. Pendências conhecidas por sobreviventes podem ser recuperadas. Uma tentativa não confirmada conhecida apenas pelo nó que caiu pode desaparecer e exigir reenvio. Uma nova falha durante essa transição fica fora da garantia inicial.

Exemplo revisado: mensagens em trânsito

A visão inicial é época $(1, 3)$, membros $\{1, 2, 3\}$ e líder 3. Os históricos já contêm a mensagem `oi`. O nó 1 origina `x`, com carimbo $(10, 1)$; o nó 3 origina `y`, com carimbo $(20, 3)$, e cai depois de enviar cópias.

Antes das pausas, o nó 1 tem todos os ACKs de `x` e a entrega. O nó 2 ainda espera o ACK do nó 1 para `x`; já recebeu o ACK do nó 3. A cópia de `y` destinada ao nó 1 chegou; a destinada ao nó 2 está em trânsito. Esse cenário respeita FIFO por par de nós: o ACK do nó 3 para `x` chegou ao nó 2 antes de sua cópia de `y`.

Estado após a pausa local	Nó 1	Nó 2
Histórico entregue	`[oi, x]`	`[oi]`
Pendências conhecidas	`[y]`	`[x]`

O Bully escolhe o nó 2 entre os sobreviventes. Ambos pausam e informam os estados acima. Se a cópia antiga de `y` chegar ao nó 2 depois de sua pausa, será descartada nesse caminho normal. Seu conteúdo, porém, já foi preservado no relato do nó 1 e será recuperado pela transição.

A união ordenada dos relatos produz `[oi, x, y]`, que estende os dois históricos. A proposta instala a visão de época $(2, 2)$, membros $\{1, 2\}$, líder 2 e esse histórico. Após os dois READYs, a retomada entrega apenas `y` aos clientes do nó 1 e `x, y` aos do nó 2. As pendências recuperadas são resolvidas pela instalação comum, sem esperar ACK do membro removido. A instalação pode recuperar uma mensagem que ainda não tinha sido confirmada; ausência de confirmação não significa cancelamento.

Uma mensagem `z` conhecida somente pelo nó 3 que caiu pode não aparecer na coleta. Ela não poderia ter sido confirmada pelo fluxo normal sem ACK dos demais membros. Seu cliente poderá repetir a tentativa com o mesmo UUID. Este exemplo não cobre falhas adicionais durante a reorganização.

Reentrada

O nó retorna sincronizando e envia `JOIN`. Sua admissão usa a mesma pausa. Recebe `SNAPSHOT` com histórico, UUIDs entregues, visão e relógio antes de entrar no grupo ativo. Depois pode participar de uma eleição Bully, inclusive se tiver o maior ID.

4 Contrato v1

Rotas. Entre nós: `POST /interno/pacote`, corpo JSON com o envelope. Clientes: WebSocket em `/ws`. Observação: `GET /estado` e `GET /historico`. Época é a lista `[contador, coordenador]`, ou `null` enquanto o nó não tem visão. Todos os pacotes levam `tipo`, `pacote_id`, `epoca` e `origem`.

Tipo	Campos além do envelope e função
`MSG`	`relogio` (carimbo original), `mensagem` {id, sala, autor, texto}.
`ACK`	`relogio` (do emissor), `evento` {origem, relogio}; época do envelope.
`HEARTBEAT`	`estado`, `coordenador`. Fora do relógio lógico.
`ELECTION` / `OK`	Sem campos extras. Iniciar eleição e responder como ID maior.
`COORDINATOR`	Sem campos extras: o vencedor é a `origem`. Não libera envios.
`VIEW_INSTALL`	`transicao`, `lider`, `membros`, `historico`, `relogio`.
`VIEW_READY`	`transicao`: confirma a instalação.
`VIEW_RESUME`	`transicao`: libera a operação após todos os READY.
`VIEW_PREPARE`	A definir em T05: transição e membros propostos.
`VIEW_STATE`	A definir em T05: histórico, pendências e relógio após o corte.
`JOIN` / `SNAPSHOT`	A definir em T06: admissão e transferência de estado.

A referência de transição (`transicao`) é a época proposta.

Respostas HTTP. `200`: processado, repetido (`duplicado`), descartado por época antiga ou ignorado; em todos os casos, o retry termina. `503`: o receptor ainda não pode processar (sem visão, aguardando `VIEW_RESUME` ou época mais nova que a sua); o canal repete o mesmo pacote sem avançar, preservando o FIFO. Falha de transporte, timeout e outros 5xx também repetem o mesmo pacote. `400`: pacote malformado, recusado antes de qualquer efeito. O envelope e os campos do tipo são conferidos primeiro; números são inteiros JSON, sem booleanos, e só os controles aceitam época `null`. Uma recusa como essa indica defeito: o pacote não é repetido, e o canal daquele destino para nele, registrando o motivo, sem deixar o seguinte ultrapassá-lo. Os outros destinos seguem. Só MSG e ACK passam pelo canal FIFO com retry e pela deduplicação por `pacote_id`. O controle é enviado uma vez, de forma direta, e é idempotente: quem precisa de resposta o reenvia a cada ciclo de heartbeat.

Bootstrap. Todos começam *sincronizando*, sem visão. O nó de maior ID configurado propõe a visão $(1, \text{ID})$ com todos os nós configurados quando recebeu heartbeat recente de cada um e nenhum tem visão. Ele envia `VIEW_INSTALL`; cada nó instala, responde `VIEW_READY` e segue pausado; com todos prontos, o coordenador retoma e envia `VIEW_RESUME`, reenviando até ver todos ativos pelos heartbeats. Logo, o grupo inicial exige os três nós. Um nó que reinicia encontra pares com visão, não faz bootstrap e aguarda admissão por `JOIN` (T06). Reiniciar todos começa um grupo novo e vazio.

Cliente. O cliente envia `{id, sala, autor, texto}`. O nó envia, na conexão, `{tipo: historico, no, mensagens}`. A cada entrega, envia `entregue` com `posicao`, `epoca`, `relogio`, `origem` e `mensagem`; esse aviso é a confirmação. Respostas imediatas: `aceita` (em andamento, não é confirmação), `confirmada` (UUID já entregue) ou `recusada` com `motivo` (campos inválidos, UUID reutilizado com outro conteúdo ou nó fora do estado ativo). Tentativa pendente ou recusada é reenviada com o mesmo UUID.

5 Decisões ainda abertas e validação

Para T05 e T06: implementação da pausa local, formato de `VIEW_PREPARE`, `VIEW_STATE`, `JOIN` e `SNAPSHOT`, reconciliação de pendências, encerramento dos canais para membros removidos e limpeza dos identificadores de pacotes processados, hoje guardados sem limite durante a execução.

Validação da v1 (`tests/`): uma simulação em memória intercala canais ao acaso, mantém FIFO em cada canal, repete pacotes e responde 503 a um nó ainda pausado. Em 200 sementes, os três nós entregaram a mesma sequência, sem perdas nem duplicatas. Defeitos injetados de propósito (entregar com ACKs parciais ou ordenar pelo relógio do receptor) fizeram a simulação falhar. Testes assíncronos cobrem Bully, suspeita, bootstrap e a exclusão de nós sincronizando. Um teste de integração sobe três processos reais em `localhost`, envia 45 mensagens concorrentes por três clientes WebSocket e derruba o líder; os sobreviventes elegeram o nó 2 e mantiveram os envios pausados. Em 6/11, testes do canal FIFO real com HTTP simulado cobriram 500, 503, resposta perdida e recusa permanente, e 37 pacotes malformados foram recusados sem efeitos. No Compose, os três nós entregaram as mesmas 60 mensagens concorrentes e, após a queda do nó 3, elegeram o nó 2 e permaneceram pausados. Esses testes não provam a correção geral.

O exemplo da seção 3 foi conferido com um modelo de dados em memória: união ordenada, prefixos, sufixos a entregar e rejeição de prefixo incompatível. Isso não executou o protocolo de reorganização, ainda não implementado.

Referências iniciais

Enunciado do professor: Trabalho Final – Sistemas Distribuídos 2026, fornecido durante o planejamento. Lamport (1978); Garcia-Molina (1982); Van Renesse et al., comunicação e mudança de visão.